Федеральное государственное образовательное бюджетное учреждение высшего образования
«ФИНАНСОВЫЙ УНИВЕРСИТЕТ ПРИ ПРАВИТЕЛЬСТВЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»
(Финансовый университет)
Кафедра информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе по дисциплине «Проектирование информационных систем»
на тему: «Проектирование информационной системы центра повышения квалификации»

Выполнил: студент группы ДПИ23-1
Маричук Михаил Борисович
Научный руководитель: Штрекер Евгений Николаевич

Москва — 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ
Глава 1. Анализ предметной области и постановка задачи
1.1. Описание предметной области центра повышения квалификации
1.2. Формулировка требований к информационной системе
1.3. Определение бизнес-логики системы
1.3.1. Управление программами и контингентом слушателей
1.3.2. Обработка заявок, проверка документов и управление доступом
1.3.3. Управление обучением, аттестацией и выдачей удостоверений
1.3.4. Административные функции управления и контроля
Глава 2. Проектирование системы
2.1. Архитектура системы
2.1.1. Разделение на клиентский, серверный и слой данных
2.1.2. Выбор технологий и обоснование проектных решений
2.2. Функциональное моделирование процессов (IDEF0, IDEF3, DFD)
2.3. Проектирование ролей и прав доступа
2.4. Концептуальная модель данных и схема БД (IDEF1X)
2.5. Проектирование надежности и безопасности
Глава 3. Реализация системы
3.1. Настройка окружения разработки и развертывание
3.2. Реализация серверной части приложения
3.3. Реализация пользовательских интерфейсов
3.4. Реализация регламентных переходов и проверок
3.5. Обработка ошибок и защита данных
Глава 4. Тестирование и проверка работоспособности системы
4.1. Методика тестирования функциональности системы
4.2. Тестирование пользовательского интерфейса
4.3. Проверка корректности основных сценариев
4.4. Проверка требований к устойчивости и безопасности
ЗАКЛЮЧЕНИЕ
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ
ПРИЛОЖЕНИЯ
Приложение А. Матрица прав доступа по ролям
Приложение Б. Сопоставление требований и компонентов системы
Приложение В. Ссылки на программный проект
Приложение Г. Диаграммы
Приложение Д. Скриншоты системы

ВВЕДЕНИЕ

В условиях цифровизации образовательной среды и роста спроса на дополнительное профессиональное образование центры повышения квалификации сталкиваются с необходимостью формализации и автоматизации ключевых процессов. Практика показывает, что при отсутствии единого цифрового контура возникают типовые проблемы: несогласованность статусов заявок, дублирование данных, задержки при проверке документов, сложность межролевого взаимодействия и низкая прозрачность управленческих показателей.

Для устранения указанных проблем требуется информационная система, поддерживающая полный жизненный цикл слушателя: от выбора программы и подачи заявки до аттестации и выдачи удостоверения. Такая система должна обеспечивать единое хранилище данных, строгую ролевую модель, контролируемые бизнес-правила и удобные интерфейсы для всех участников процесса.

Объектом исследования является процесс организации и сопровождения обучения в центре повышения квалификации.

Предметом исследования являются методы и средства проектирования и реализации информационной системы центра повышения квалификации на основе трехзвенной архитектуры.

Целью курсовой работы является проектирование и реализация информационной системы центра повышения квалификации, обеспечивающей управление программами, заявками, оплатой, обучением, аттестацией и выдачей удостоверений в едином цифровом контуре.

Для достижения поставленной цели решаются следующие задачи:

1. Провести анализ предметной области и выделить ключевые бизнес-процессы.
2. Сформировать функциональные и нефункциональные требования к системе.
3. Выполнить проектирование системы с использованием нотаций IDEF0, IDEF3, DFD, UML и IDEF1X.

4. Разработать модель данных и обосновать ее нормализацию до третьей нормальной формы.
5. Реализовать программное решение на базе Java/Spring Boot.
6. Обеспечить требования безопасности, устойчивости и ролевого разграничения доступа.
7. Провести тестирование основных сценариев и подтвердить работоспособность системы.

Практическая значимость работы заключается в том, что разработанная система может использоваться как пилотный промышленно-ориентированный контур цифровой платформы центра повышения квалификации и служить основой для дальнейшего функционального расширения.

Глава 1. Анализ предметной области и постановка задачи

1.1. Описание предметной области центра повышения квалификации

Центр повышения квалификации представляет собой образовательную организацию, в которой параллельно ведутся наборы по нескольким программам, осуществляется обработка заявок, проверка документов слушателей, контроль оплаты, сопровождение учебного процесса и оформление итоговых документов. В процесс вовлечены различные категории пользователей, для каждой из которых характерны собственные действия и зона ответственности.

Слушатель выбирает образовательную программу, подает заявку, предоставляет документы, получает доступ к обучению, проходит аттестацию и ожидает выдачи удостоверения. Методист осуществляет проверку документов, принимает решения по допуску и управляет очередями заявок. Преподаватель ведет группу, фиксирует прогресс, вносит результат аттестации и завершает обучение. Администратор контролирует работу системы, отслеживает статистику и принимает управленческие решения.

Ключевой особенностью предметной области является наличие последовательного бизнес-процесса с жесткими условиями перехода между статусами. Нарушение этого порядка приводит к операционным ошибкам:

например, открытие полного доступа без подтвержденных документов или попытка выдачи удостоверения без успешной аттестации. Следовательно, система должна не только хранить данные, но и обеспечивать соблюдение правил предметной области на уровне бизнес-логики.

1.2. Формулировка требований к информационной системе

На основе анализа предметной области сформулированы требования к информационной системе.

К функциональным требованиям относятся:

1. Регистрация и авторизация пользователей.
2. Ролевая модель доступа с ролями ADMIN, METHODIST, TEACHER, STUDENT.
3. Управление программами (создание, изменение, удаление, просмотр).
4. Подача и сопровождение заявок слушателей.
5. Проверка документов и управление статусами допуска.
6. Управление оплатой и синхронизация статуса заявки после оплаты.
7. Управление учебным прогрессом и итоговой аттестацией.
8. Выдача удостоверений при соблюдении условий допуска.
9. Поиск, фильтрация и сортировка данных в табличных представлениях.
10. Формирование статистики и управленческой аналитики.
11. Наличие отдельной страницы «Об авторе».
12. Обработка ошибок с информативной обратной связью пользователю.

К нефункциональным требованиям относятся:

1. Безопасность хранения учетных данных и разграничения прав.
2. Устойчивость к ошибочному вводу и исключительным ситуациям.
3. Приемлемая производительность при типовых операциях CRUD и фильтрации.
4. Поддерживаемость за счет многослойной архитектуры.
5. Воспроизводимое развертывание в локальной и облачной среде.

1.3. Определение бизнес-логики системы

Бизнес-логика системы строится вокруг линейного процесса:

Программа -> Заявка -> Оплата -> Аттестация -> Удостоверение.

Каждый этап имеет формальные условия перехода, отдельный набор ролей и явный результат в доменной модели. Такой подход позволяет исключить двусмысленность в работе персонала и повысить прозрачность состояния конкретного слушателя.

1.3.1. Управление программами и контингентом слушателей

Система обеспечивает централизованное ведение реестра образовательных программ. Для каждой программы задаются название, описание, тип, категория, длительность, период реализации и признак активности. Доступ к изменению программ предоставляется сотрудникам бэк-офиса (методисту и администратору), а просмотр доступен всем пользователям.

Слушатель взаимодействует с программами через публичный каталог, после чего инициирует переход к подаче заявки. В результате формируется единая связка «слушатель — программа — заявка», являющаяся отправной точкой дальнейшего процесса.

1.3.2. Обработка заявок, проверка документов и управление доступом

После создания слушателем заявка получает стартовое состояние: заявка подана, документы ожидают проверки, доступ к учебному контуру закрыт.

Далее заявка попадает в операционный контур методиста. На этом этапе методист:

1. проверяет комплектность и корректность документов;
2. фиксирует результат проверки (подтверждение или отклонение);
3. назначает преподавателя для дальнейшего учебного маршрута;
4. управляет режимом доступа (пробный/полный) в зависимости от оплаты и текущего состояния заявки.

При отклонении документов заявка переводится в состояние отказа, доступ закрывается, назначение преподавателя снимается. При подтверждении документов заявка переводится на этап допуска и оплаты.

Система поддерживает два режима доступа:

1. Пробный доступ сроком 3 дня.

2. Полный доступ при выполнении обязательных условий.

Правила доступа:

1. пробный режим открывается только после подтверждения документов и до полной оплаты;
2. при открытии пробного режима фиксируется дата его окончания;
3. если пробный период истек и платеж не подтвержден, доступ автоматически закрывается, а заявка возвращается в рабочий контур методиста;
4. полный доступ открывается только после подтверждения документов, оплаты и назначения преподавателя.

Это правило проверяется в серверной логике и не зависит от действий пользователя в интерфейсе.

1.3.3. Управление обучением, аттестацией и выдачей удостоверений

После перехода в учебный контур преподаватель получает возможность обновлять прогресс слушателя и вносить результат аттестации. Для аттестации используется порог успешности 75 баллов. При значении ниже порога система переводит заявку в состояние продолжения обучения. При успешной аттестации заявка может быть завершена только после подтверждения завершения со стороны преподавателя.

Выдача удостоверения реализована как отдельный этап, доступный методисту и администратору. Система выполняет обязательную проверку условий допуска к выпуску удостоверения: успешная аттестация и завершение обучения преподавателем. Таким образом исключается преждевременное оформление итогового документа.

1.3.4. Административные функции управления и контроля

Администратор работает с отдельной аналитической панелью, которая выполняет три задачи:

1. Операционный контроль очередей (документы, подтвержденные но неоплаченные заявки, пробный доступ, активное обучение, готовность к выдаче удостоверений).

2. Управленческий мониторинг нагрузки и узких мест (воронка, bottleneck-блок, риск-приоритеты).
3. Навигация в drilldown-режим: от агрегированного KPI к карточке конкретной заявки.

Логика админского доступа построена по принципу «чтение + управление критическими решениями». Администратор видит все контуры сразу, но фактические изменения выполняются через те же бизнес-сервисы и проверки, что и у профильных ролей (методист/преподаватель). Это исключает «обходные» административные действия и сохраняет единые правила предметной области.

Важно, что здесь нет противоречия с ролевым разграничением. Администратор имеет расширенный доступ к разделам и аналитике, но каждое изменение проходит через те же регламентные проверки, что и у профильных ролей. Поэтому «полный доступ» означает широкий охват контуров, а не обход обязательных правил процесса.

Структурно дашборд включает:
1. KPI-плитки для быстрого контроля состояния процесса.
2. Воронку этапов заявки для оценки конверсии между шагами.
3. Таблицу проблемных кейсов с фильтрами по риску, роли и типу очереди.
4. Drilldown-маршруты в операционные разделы, где доступна предметная детализация.

Глава 2. Проектирование системы

2.1. Архитектура системы

2.1.1. Разделение на клиентский, серверный и слой данных

В проекте выбрана трехзвенная архитектура:
1. Клиентский слой — браузерный пользовательский интерфейс на HTML/Thymeleaf.
2. Серверный слой — приложение Spring Boot, реализующее бизнес-логику, маршрутизацию и безопасность.

3. Слой данных — реляционная база данных (PostgreSQL в прод-контуре, H2 в локальной разработке).

Такое разделение обеспечивает низкую связанность модулей, упрощает сопровождение и создает основу для масштабирования. Все критичные правила предметной области реализованы в серверном слое и выполняются независимо от типа клиента.

2.1.2. Выбор технологий и обоснование проектных решений

Для реализации серверной части использован Java 17 и Spring Boot 3.5.0. Выбор обусловлен зрелостью экосистемы, наличием встроенных механизмов безопасности и удобством реализации многослойной архитектуры.

Для слоя данных выбран Spring Data JPA (Hibernate), что позволяет работать с сущностями предметной области на уровне объектной модели и снижает вероятность SQL-инъекций за счет параметризованного доступа к данным.

Для аутентификации и авторизации использован Spring Security с хранением паролей в BCrypt. Этот подход соответствует требованиям безопасности и упрощает реализацию ролевого разграничения доступа.

Для интерфейсной части выбран серверный рендеринг Thymeleaf. Такое решение обеспечивает единый технологический контур, прозрачную трассировку изменений от доменной модели до представления и предсказуемый эксплуатационный профиль.

При выборе технологического стека ключевыми критериями были архитектурная целостность, трассируемость требований и предсказуемость эксплуатации. Поэтому выбран единый веб-контур на Spring Boot/Thymeleaf без выделения отдельного SPA-клиента и независимого API-контура: такое решение обеспечивает явную связку «модель -> бизнес-логика -> интерфейс», единый механизм безопасности и согласованный релизный цикл.

Принятое решение обеспечивает инженерную целостность: критичные правила реализуются в серверном слое, интерфейс получает уже валидированное состояние процесса, а сопровождение системы остается

предсказуемым при расширении функциональности.

2.2. Функциональное моделирование процессов (IDEF0, IDEF3, DFD)

Проектирование процессов выполнено в обязательных нотациях.

IDEF0 A-0 описывает верхнеуровневую функцию «Управление обучением в ЦПК». Входами являются заявки, документы, данные оплаты и результаты аттестации; управляющими воздействиями — регламенты и правила допуска; механизмами — пользователи и информационная система; выходами — статусы, отчеты и выданные удостоверения.

IDEF0 A0 декомпозирует процесс на пять функций:

1. Управление программами.
2. Управление заявками и документами.
3. Управление оплатой и доступом.
4. Управление обучением и аттестацией.
5. Выдача удостоверений и аналитика.

IDEF3 фиксирует последовательность событий и развилки по условиям «документы подтверждены/отклонены», «оплата выполнена/не выполнена», «аттестация пройдена/не пройдена».

DFD отражает потоки данных между внешними сущностями (слушатель, методист, преподаватель, администратор), процессами системы и хранилищами users, programs, applications, payments, assessment_results, certificates.

IDEF3 и UML Activity визуально близки, но описывают разные уровни модели: IDEF3 фиксирует бизнес-события и статусные развилки заявки, а UML Activity показывает пользовательский маршрут от лица слушателя. Такое разделение позволяет раздельно проверять процессные переходы и пользовательские сценарии.

В UML-диаграммах сохранены понятные предметные наименования сценариев и ролей; подписи акторов, сценариев и пояснений приведены на русском языке.

Все диаграммы согласованы между собой и соответствуют реализованным в системе процессам и данным.

Для удобства чтения ниже приведены краткие пояснения по назначению каждой диаграммы:

1. IDEF0 A-0 — показывает границы системы, входы, выходы, механизмы и управляющие воздействия.
2. IDEF0 A0 — раскрывает основные блоки процесса и их взаимосвязи.
3. IDEF0 A2 — детализирует критичный контур обработки заявок и допуска.
4. IDEF3 — фиксирует фактическую последовательность событий и развилки по условиям.
5. DFD — показывает движение данных между ролями, процессами и хранилищами.
6. UML Use Case — определяет, какие функции доступны каждой роли.
7. UML Activity — отражает пользовательский путь по этапам процесса.
8. UML Sequence — показывает порядок взаимодействия между компонентами в ключевом сценарии.
9. IDEF1X/ER — описывает структуру данных и связи сущностей в базе.

2.3. Проектирование ролей и прав доступа

Ролевая модель построена по принципу минимально необходимых полномочий:

1. STUDENT — только собственные данные и действия по своей образовательной траектории.
2. METHODIST — верификация документов, маршрутизация заявок, выпуск удостоверений.
3. TEACHER — управление назначенными группами, прогрессом и аттестацией.
4. ADMIN — полный доступ и аналитический контроль.

Разграничение реализовано на двух уровнях:

1. уровень входа в разделы системы;
2. уровень отдельных операций внутри разделов.

Двухуровневое разграничение доступа (маршруты и методы) сохраняет корректность прав при расширении функциональности.

Таким образом, роли определяют «кто и куда может зайти», а регламентные проверки определяют «что именно можно сделать в текущем

состоянии заявки».

Дополнительно роли привязаны к этапам жизненного цикла заявки, а не только к URL-маршрутам. Это означает, что операция разрешается не по факту наличия роли как таковой, а по сочетанию «роль + допустимый этап процесса + состояние заявки». Такой подход снижает вероятность логических ошибок при ручной работе сотрудников.

В результате модель доступа поддерживает не только безопасность, но и процессную дисциплину: действия пользователей становятся воспроизводимыми, проверяемыми и согласованными с регламентом ЦПК.

2.4. Концептуальная модель данных и схема БД (IDEF1X)

Система включает шесть ключевых сущностей: Пользователь, Программа, Заявка, Оплата, Аттестация, Удостоверение.

Центральной сущностью выступает Заявка, связывающая слушателя, программу, преподавателя, оплату, аттестацию и удостоверение. Данное решение обеспечивает прослеживаемость жизненного цикла по одной бизнес-единице.

Основные отношения:

1. Program 1:M Application.
2. User(STUDENT) 1:M Application.
3. User(TEACHER) 1:M Application (по назначению).
4. Application 1:1 Payment.
5. Application 1:1 AssessmentResult.
6. Application 1:1 Certificate.

Обозначение 1:M означает связь «один ко многим»: одному объекту первого типа соответствует несколько объектов второго типа.

Обозначение 1:1 означает связь «один к одному»: одному объекту соответствует ровно один связанный объект.

Нормализация до 3НФ обеспечивается разделением независимых сущностей по отдельным таблицам и отсутствием транзитивных зависимостей между неключевыми атрибутами.

В практическом смысле 3НФ означает, что в каждой таблице хранятся только данные одной сущности, а атрибуты зависят от ключа этой таблицы напрямую. Это снижает дублирование и уменьшает риск противоречивых данных.

При проектировании модели данных ключевым решением стало выделение заявки как единой точки оркестрации процесса. Ключевые этапы процесса хранятся в рамках одной бизнес-сущности, что позволяет однозначно интерпретировать текущее состояние слушателя без «склейки» разрозненных таблиц.

Связи типа 1:1 для платежа, аттестации и удостоверения выбраны как контроль целостности предметной области: на одну заявку допускается один актуальный платежный результат, один актуальный итог аттестации и один итоговый документ. Это упрощает контроль корректности данных и снижает риск дублирующих записей в операционном контуре.

2.5. Проектирование надежности и безопасности

Для надежности применена централизованная обработка исключений и отдельная логика маршрутизации ошибок. Вместо аварийного завершения пользователь получает контролируемую страницу с понятным сообщением.

Для безопасности реализованы:

1. хранение паролей в зашифрованном виде;
2. сессионная аутентификация;
3. ролевые ограничения доступа;
4. безопасная работа с данными через ORM;
5. безопасный вывод данных в пользовательском интерфейсе.

В отличие от типового прототипного подхода, критичные данные предметной области не хранятся в localStorage как источник истины. Источником истины является сервер и реляционная база данных.

Проектирование надежности рассматривалось как отдельная задача, а не побочный эффект реализации. В фокусе находились три сценария отказа: некорректный пользовательский ввод, нарушение прав доступа и исключительные ситуации доменной логики. Для каждого сценария

сформирован предсказуемый ответ системы, что исключает «немые» ошибки и повышает управляемость эксплуатации.

Таким образом, блок безопасности и устойчивости встроен в архитектуру на уровне проектных решений и напрямую поддерживает требования к качеству итоговой системы.

Глава 3. Реализация системы

3.1. Настройка окружения разработки и развертывание

Сборка и запуск проекта выполняются с использованием Maven. Для локальной разработки используется профиль dev и встроенная база H2, что позволяет запускать систему без внешней инфраструктуры. Для облачного контура предусмотрен профиль render и PostgreSQL.

В проекте реализован Docker-контур:

1. multi-stage Dockerfile для сборки и запуска приложения;
2. render.yaml для декларативного развертывания web-сервиса и базы данных;
3. health-check по маршруту /login.

Локальный операционный запуск автоматизирован скриптами start-local.sh, status-local.sh, stop-local.sh, что обеспечивает воспроизводимый цикл запуска, контроля состояния и остановки сервиса.

Разделение профилей dev и render использовано для сохранения паритета логики между локальным и облачным контурами: различается только инфраструктурная конфигурация (тип БД, параметры окружения), но не бизнес-поведение приложения. Это позволяет корректно переносить результаты тестирования из локальной среды в облачный контур.

Контур развертывания стандартизует операции запуска, проверки состояния и остановки сервиса; доступность приложения контролируется через явный health-check.

3.2. Реализация серверной части приложения

Серверная часть реализована по многослойной модели:

1. слой предметных сущностей;

2. слой доступа к данным;
3. слой бизнес-правил и аналитики;
4. слой веб-взаимодействия с пользователем;
5. слой конфигурации безопасности и инициализации.

Функциональные модули серверной части:
1. модуль программ (каталог и операции управления);
2. модуль заявок (создание, сопровождение, статусные переходы);
3. модуль методиста (проверка документов, управление доступом, назначение преподавателей);
4. модуль преподавателя (группы, прогресс, аттестация);
5. модуль оплаты;
6. модуль выдачи удостоверений;
7. модуль административной аналитики.

Критичные переходы жизненного цикла заявки централизованы в отдельном workflow-модуле, что исключает дублирование правил в разных частях системы.

Обработка запроса организована по единому конвейеру: веб-слой -> бизнес-слой -> слой данных -> пользовательское представление. Такое построение отделяет транспортный слой от предметной логики и позволяет повторно использовать одни и те же правила в разных интерфейсных точках входа.

Принцип централизации workflow-логики критичен для данного проекта: изменение правила в одном месте автоматически распространяется на все сценарии, где это правило применяется. За счет этого снижается вероятность регрессий и расхождения поведения между экранами.

3.3. Реализация пользовательских интерфейсов

Интерфейс разделен по ролям:
1. Публичный контур: главная страница, каталог программ, консультационная форма.
2. Контур слушателя: личный кабинет, профиль, заявка, оплата, просмотр статусов.
3. Контур методиста: очереди документов и доступа, выпуск удостоверений.

4. Контур преподавателя: группы, прогресс, аттестация.
5. Контур администратора: расширенная аналитика, drilldown, case-карточка.

Табличное представление данных используется во всех операционных разделах, предусмотрены поиск, фильтрация и сортировка по релевантным полям.

При проектировании интерфейсов использован единый паттерн представления информации: «контекст -> статус -> доступные действия». В результате пользователь в любой роли сначала видит текущее состояние объекта, затем допустимые операции, и только после этого переходит к детальной форме. Это уменьшает количество ошибочных действий и ускоряет выполнение типовых операций.

Для выравнивания UX между ролями применены одинаковые принципы:
1. единая навигационная структура;
2. однородные таблицы и карточки сущностей;
3. одинаковые правила валидации и форматирования данных;
4. прозрачные сообщения о результатах действий.

Отдельно выделен кабинет преподавателя как самостоятельный рабочий контур, где совмещены управление группами, фиксация прогресса и внесение результатов аттестации. Это позволяет преподавателю работать в рамках одной операционной зоны без переключения между несвязанными страницами.

3.4. Реализация регламентных переходов и проверок

В серверном контуре реализованы регламентные проверки, управляющие переходами статусов заявки:
1. порог успешной аттестации: не менее 75 баллов;
2. пробный доступ: 3 дня с фиксацией даты окончания;
3. полный доступ: только после подтверждения документов, оплаты и назначения преподавателя;
4. завершение обучения: только после успешной аттестации;
5. выдача удостоверения: только при успешной аттестации и подтвержденном завершении обучения преподавателем.

Проверки выполняются в серверных сервисах до изменения статуса и записи в БД, поэтому корректность процесса не зависит от поведения клиентского интерфейса.

Связка правил с прикладной реализацией закреплена следующим образом:

1. проверки аттестации и завершения обучения — в контроллерах и сервисах контура преподавателя;
2. управление пробным/полным доступом и синхронизация статусов — в workflow-модуле;
3. условия выпуска удостоверения — в сервисах и контроллерах удостоверений.

Такое построение обеспечивает единый источник проверки регламентных условий и упрощает сопровождение при развитии функциональности.

3.5. Обработка ошибок и защита данных

Обработка ошибок реализована централизованно:

1. Валидационные ошибки форм возвращаются с корректным сообщением.
2. Ошибки доступа обрабатываются с возвратом 403 или редиректом на авторизацию.
3. Системные ошибки переводятся на единую страницу обработки ошибок.
4. Ошибки несуществующих маршрутов обрабатываются отдельной страницей 404.

Защита данных реализована через ролевой доступ, проверку полномочий при работе с сущностями и исключение прямого доступа слушателя к чужим заявкам.

Классификация ошибок в проекте строится по принципу «тип ошибки -> ожидаемая реакция интерфейса»: пользовательские ошибки возвращаются в форму с подсказкой исправления, ошибки прав доступа завершаются контролируемым отказом, системные исключения переводятся в безопасный шаблон без раскрытия внутренней реализации.

Такой подход одновременно решает две задачи: сохраняет удобство работы пользователя и снижает риск утечки технической информации о внутреннем устройстве системы.

Глава 4. Тестирование и проверка работоспособности системы

4.1. Методика тестирования функциональности системы

Тестирование проведено в комбинированном формате:

1. Модульные и интеграционные автотесты для критичных подсистем.
2. Сценарное ручное тестирование пользовательских потоков по ролям.
3. Негативные проверки, подтверждающие корректность отказов.

В проекте присутствуют автоматизированные тесты, покрывающие:

1. инициализацию данных;
2. ключевые контроллеры;
3. аналитический сервис;
4. базовую проверку корректности запуска приложения.

Методика тестирования ориентирована на проверку не только отдельных функций, но и целостных пользовательских маршрутов. В качестве критерия приемки использовалось выполнение трех условий: корректный переход между статусами, сохранение целостности данных и предсказуемая реакция интерфейса при ошибках.

Ручные проверки выполнялись по ролевому чек-листу:

1. сценарии слушателя (регистрация, выбор программы, подача заявки, оплата, отслеживание статуса);
2. сценарии методиста (проверка документов, назначение преподавателя, управление допуском, выпуск удостоверения);
3. сценарии преподавателя (ведение прогресса и внесение результатов аттестации);
4. сценарии администратора (операционный контроль и детализация по заявкам).

По каждому сценарию фиксировались входные условия, ожидаемый результат и фактический результат. Такой формат позволил проверить

систему не фрагментарно, а как целостный процесс.

4.2. Тестирование пользовательского интерфейса

Интерфейс проверялся на уровне основных маршрутов и операций:

1. корректность отображения навигации в зависимости от роли;
2. валидация форм регистрации, заявки, оплаты и аттестации;
3. корректность фильтров и сортировок;
4. корректность редиректов после авторизации.

Проверено, что пользователь видит только те разделы, которые соответствуют его роли, а попытка доступа к закрытым разделам корректно блокируется.

Дополнительно проверялась визуальная согласованность экранов между ролями: идентичность базовых элементов навигации, повторяемость форматов таблиц и карточек, а также единый стиль сообщений об успехе и ошибке. Такая унификация снижает когнитивную нагрузку при переходе пользователей между связанными процессами.

4.3. Проверка корректности основных сценариев

Ниже приведены ключевые сценарии приемки.

1. Сценарий «Регистрация слушателя»:

   ожидается создание новой учетной записи с ролью STUDENT.

   Результат: выполнено.

2. Сценарий «Создание заявки слушателем»:

   ожидается создание заявки со стартовыми параметрами процесса: заявка подана, документы на проверке, доступ закрыт.

   Результат: выполнено.

3. Сценарий «Подтверждение документов методистом»:

   ожидается перевод заявки на следующий этап обработки и допуска.

   Результат: выполнено.

4. Сценарий «Открытие пробного доступа методистом»:

   ожидается открытие пробного доступа с автоматической датой завершения через 3 дня.

Результат: выполнено.

5. Сценарий «Истечение trial без оплаты»:

ожидается автоматическое закрытие доступа и возврат заявки в рабочую очередь.

Результат: выполнено.

6. Сценарий «Попытка открыть полный доступ без оплаты»:

ожидается отказ операции с информативной ошибкой.

Результат: выполнено.

7. Сценарий «Оплата и синхронизация статуса заявки»:

ожидается сохранение платежа и корректное обновление этапа заявки.

Результат: выполнено.

8. Сценарий «Аттестация ниже порога»:

ожидается продолжение обучения без завершения процесса.

Результат: выполнено.

9. Сценарий «Выдача удостоверения без допуска»:

ожидается отказ операции.

Результат: выполнено.

10. Сценарий «Доступ STUDENT к /admin/dashboard-v2»:

ожидается отказ с кодом 403.

Результат: выполнено.

4.4. Проверка требований к устойчивости и безопасности

Проведены дополнительные проверки:

1. Обработка несуществующего маршрута с возвратом 404-страницы.
2. Обработка некорректного ввода данных без аварийного завершения.
3. Проверка разграничения доступа по ролям.
4. Проверка хранения паролей в хэшированном виде.
5. Проверка устойчивости бизнес-правил при повторных операциях.

Отдельно проверялись повторные и граничные действия: повторная отправка форм, попытка выполнения запрещенного перехода статуса, доступ к административным маршрутам без необходимой роли. Во всех случаях система сохраняла корректное состояние и возвращала ожидаемую реакцию

без критических сбоев.

Результаты подтверждают соответствие решения требованиям надежности и безопасности для информационной системы центра повышения квалификации.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы выполнены анализ, проектирование и реализация информационной системы центра повышения квалификации. Разработанное решение покрывает полный жизненный цикл слушателя от выбора программы до выдачи удостоверения и обеспечивает единый цифровой контур взаимодействия всех ролей.

Выполнены основные требования к курсовому проекту:

1. Реализована трехзвенная архитектура.
2. Реализованы регистрация, авторизация, ролевой доступ и ключевые CRUD-функции.
3. Обеспечены поиск, сортировка и табличное представление данных.
4. Реализованы статистические функции и административная аналитика.
5. Реализована обработка ошибок и меры базовой безопасности.
6. Подтверждена работоспособность системы тестированием.

Полученный результат обладает практической применимостью и может быть расширен в направлении интеграции с внешними платежными сервисами, уведомлениями, аудиторским контуром и экспортом отчетности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Чистов Д.В., Мельников П.П., Золотарюк А.В., Ничепорук Н.Б. Проектирование информационных систем: учебник и практикум. Москва: Юрайт, 2023.
2. Григорьев М.В., Григорьева И.И. Проектирование информационных систем: учебное пособие. Москва: Юрайт, 2023.
3. Галиаскаров Э.Г., Воробьев А.С. Анализ и проектирование систем с использованием UML. Москва: Юрайт, 2023.

4. Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/docs/current/reference/html/ (дата обращения: 10.02.2026).
5. Spring Security Reference Documentation: https://docs.spring.io/spring-security/reference/ (дата обращения: 10.02.2026).
6. Hibernate ORM Documentation: https://hibernate.org/orm/documentation/ (дата обращения: 10.02.2026).
7. PostgreSQL Documentation: https://www.postgresql.org/docs/ (дата обращения: 10.02.2026).
8. Thymeleaf Documentation: https://www.thymeleaf.org/documentation.html (дата обращения: 10.02.2026).
9. Docker Documentation: https://docs.docker.com/ (дата обращения: 10.02.2026).
10. OWASP Top 10 Web Application Security Risks: https://owasp.org/www-project-top-ten/ (дата обращения: 10.02.2026).
11. Методические указания по выполнению курсовой работы (локальный документ).
12. Критерии оценки курсовой работы (локальный документ).

ПРИЛОЖЕНИЯ

Приложение А. Матрица прав доступа по ролям

Функция/раздел	STUDENT	METHODIST	TEACHER	ADMIN
Просмотр программ	просмотр	просмотр	просмотр	просмотр
Создание заявки	создание	нет	нет	нет
Проверка документов	нет	управление	нет	управление
Назначение преподавателя	нет	управление	нет	управление
Управление платежами	частично (checkout)	управление	нет	управление
Прогресс обучения	просмотр	просмотр	управление	управление
Аттестация	просмотр	просмотр	управление	управление
Выдача удостоверения	просмотр статуса	управление	нет	управление
Аналитика KPI	нет	ограниченно	ограниченно	полный доступ

Приложение Б. Сопоставление требований и компонентов системы

Требование	Компонент
Авторизация и роли	Модуль аутентификации и ролевого доступа
Управление заявками	Модуль жизненного цикла заявок
Очереди методиста	Операционный модуль проверки документов и допуска
Учебная нагрузка и аттестация	Модуль преподавателя и аттестации
Оплата	Финансовый модуль
Удостоверения	Модуль выпуска итоговых документов
Аналитика	Модуль управленческой и операционной аналитики
Устойчивость ошибок	Модуль централизованной обработки ошибок

Приложение В. Ссылки на программный проект

1. Репозиторий исходного кода: https://github.com/mi4gang/cpk-is-kursovaya
2. Развернутая версия системы (Render): https://cpk-is-web.onrender.com/

Приложение Г. Диаграммы

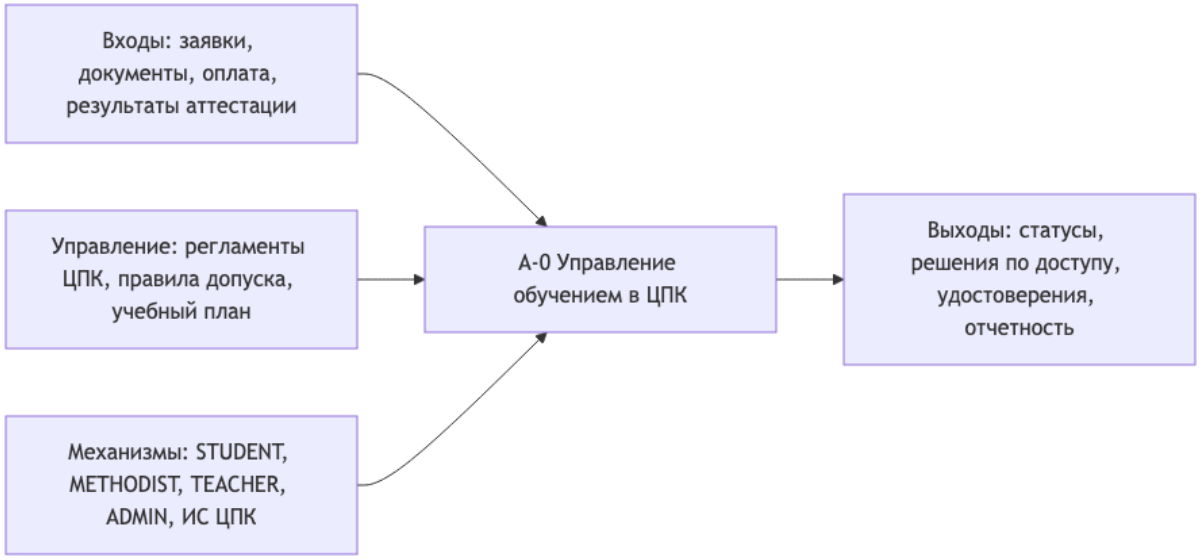

Рисунок 1 — IDEF0 A-0: Управление обучением в ЦПК

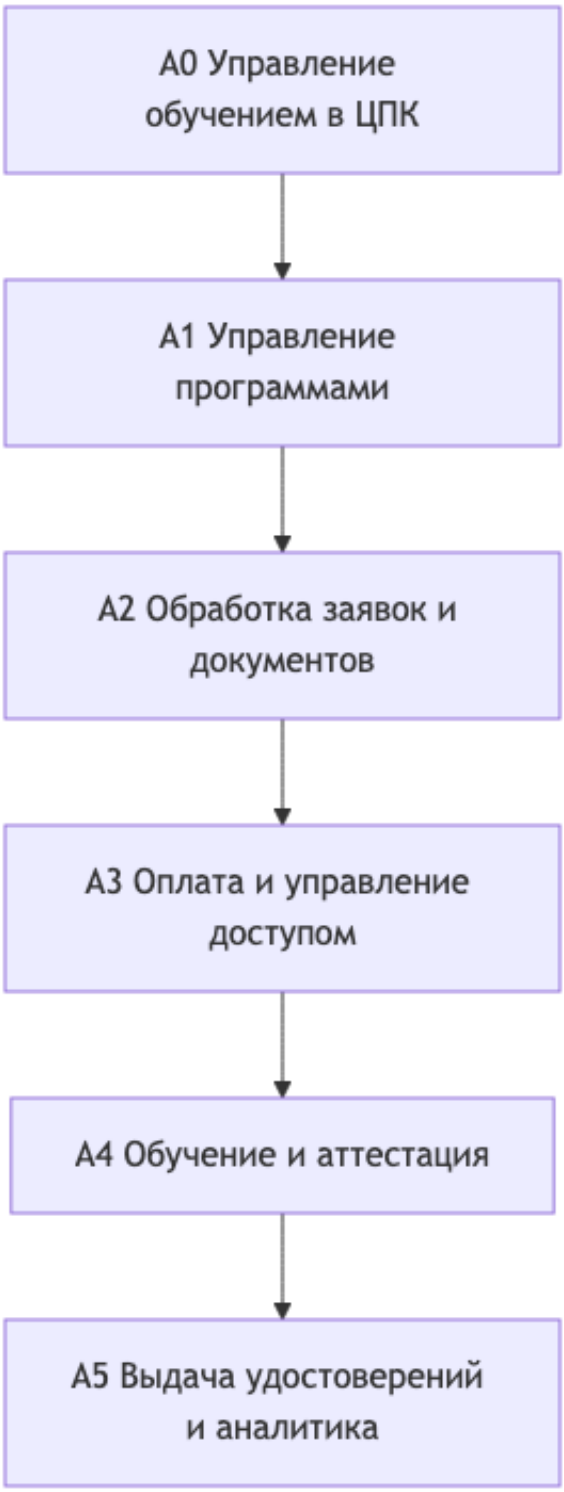

Рисунок 2 — IDEF0 A0: Декомпозиция верхнего процесса

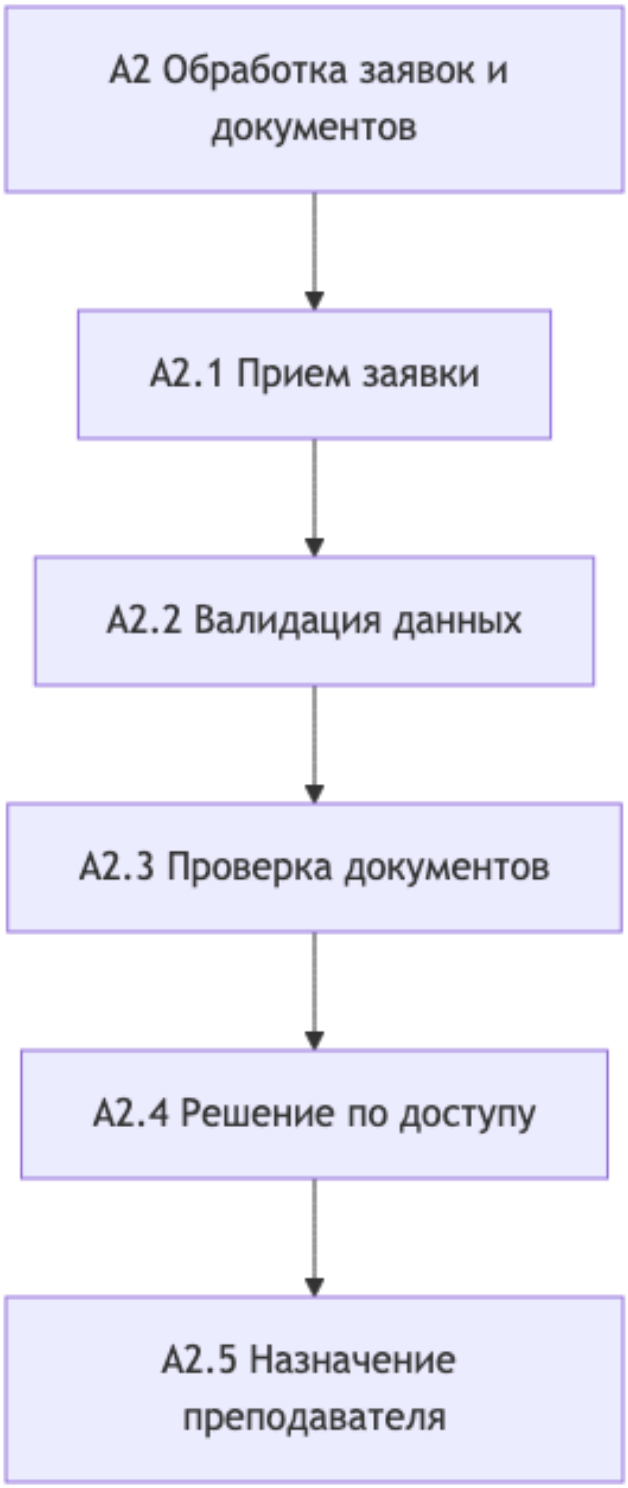

Рисунок 3 — IDEF0 A2: Декомпозиция обработки заявок

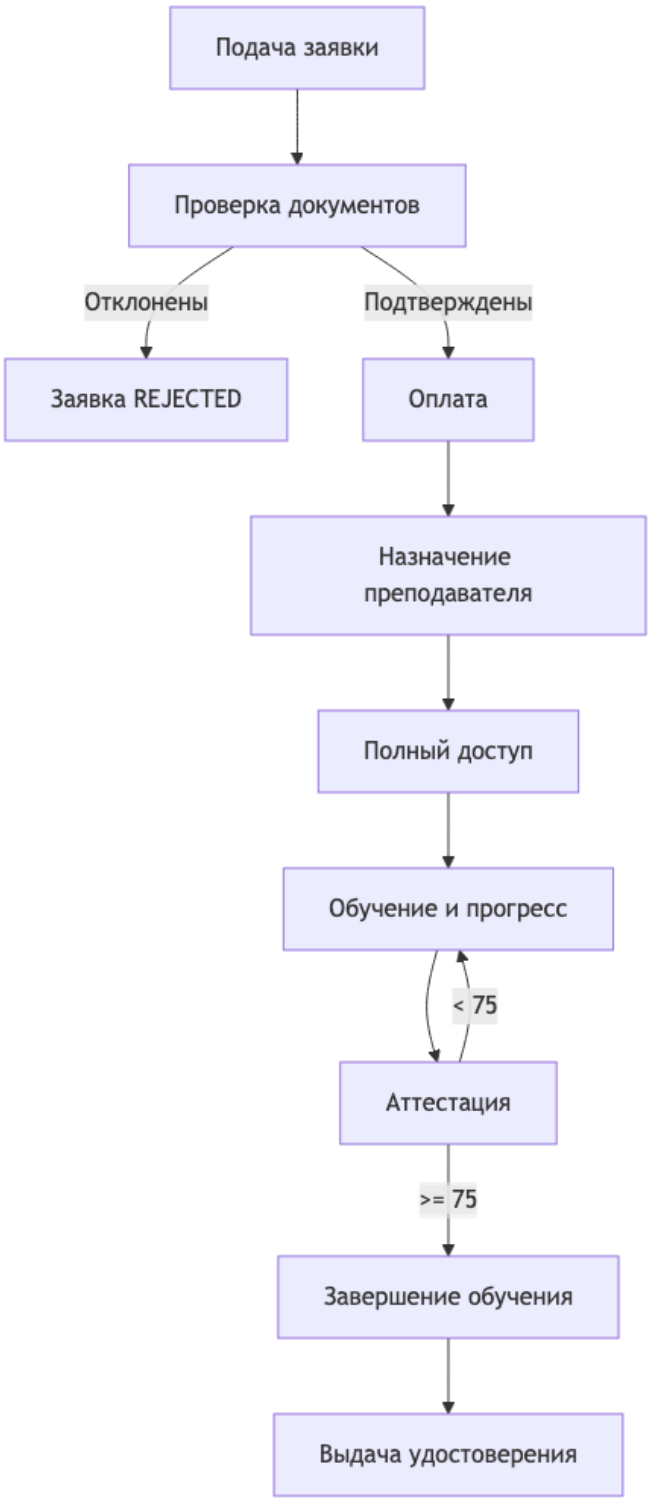

Рисунок 4 — IDEF3: Сценарий жизненного цикла заявки

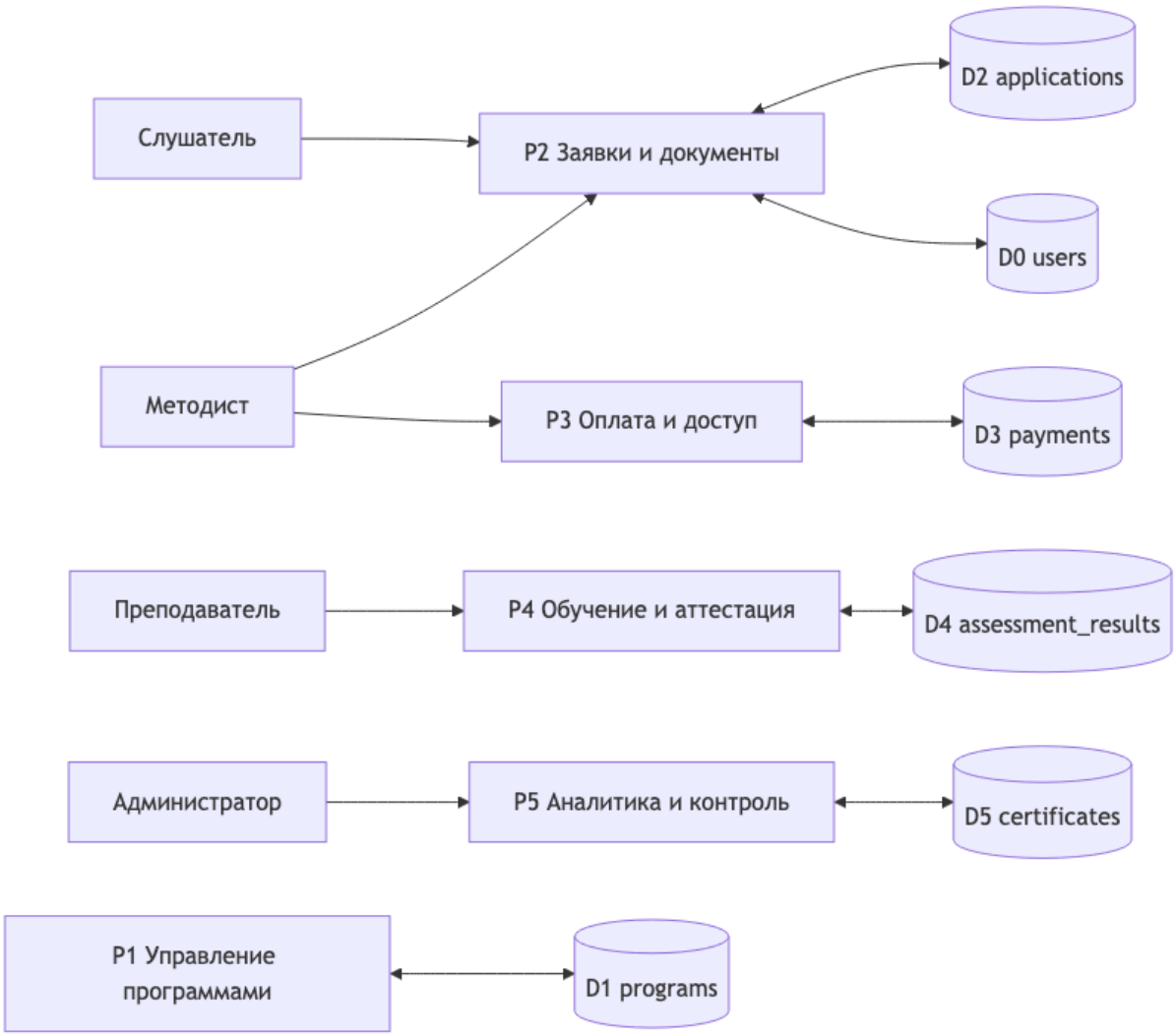

Рисунок 5 — DFD (логический уровень): основные потоки данных

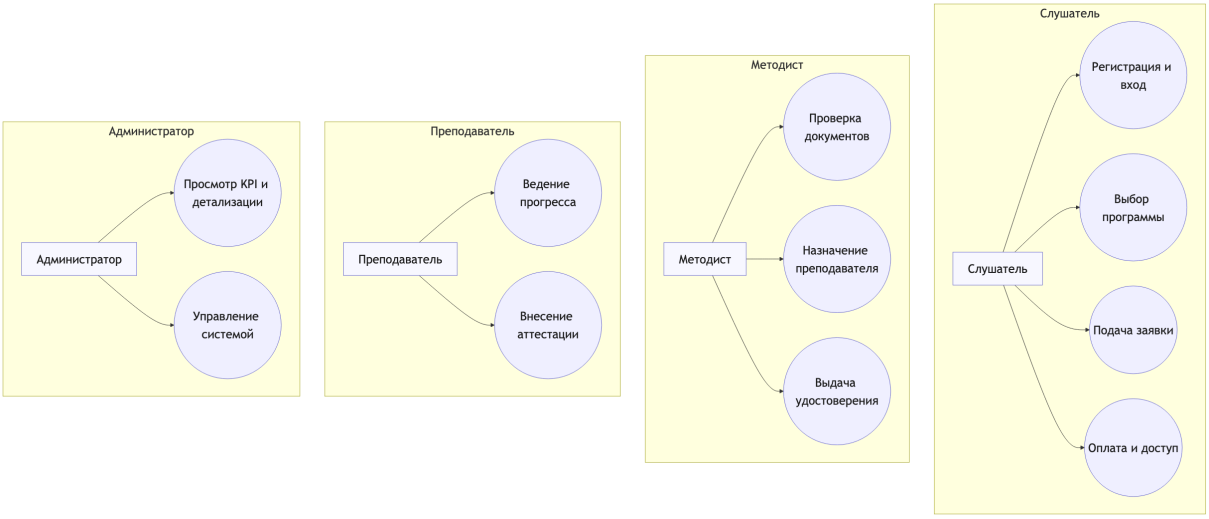

Рисунок 6 — UML Use Case (диаграмма вариантов использования): акторы и прецеденты

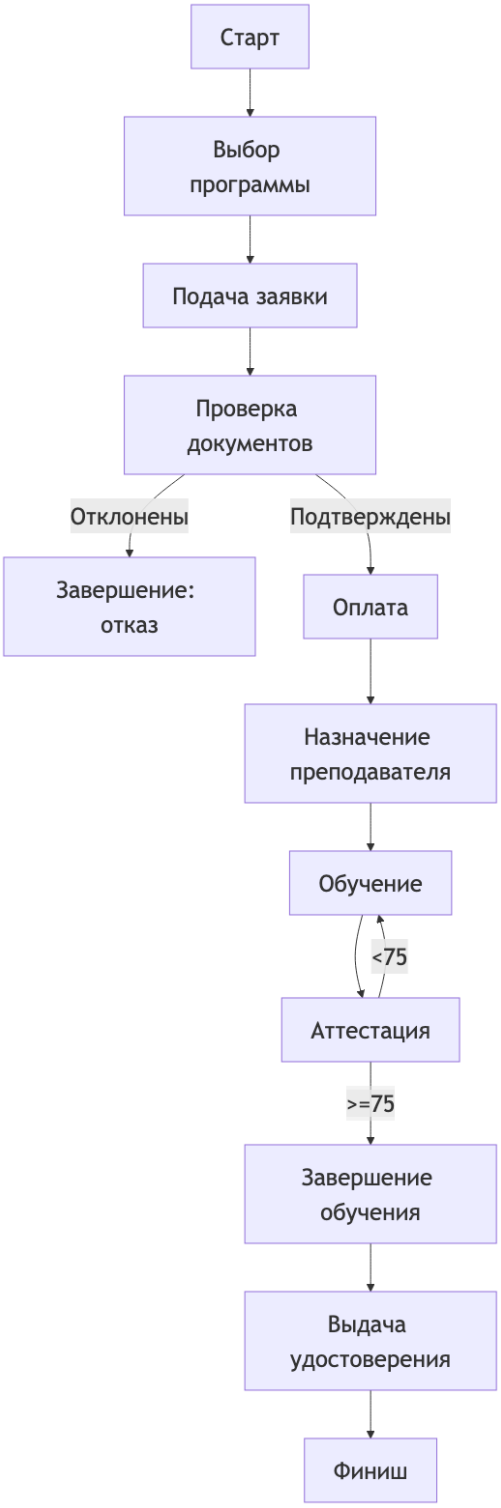

Рисунок 7 — UML Activity (диаграмма активности): путь слушателя

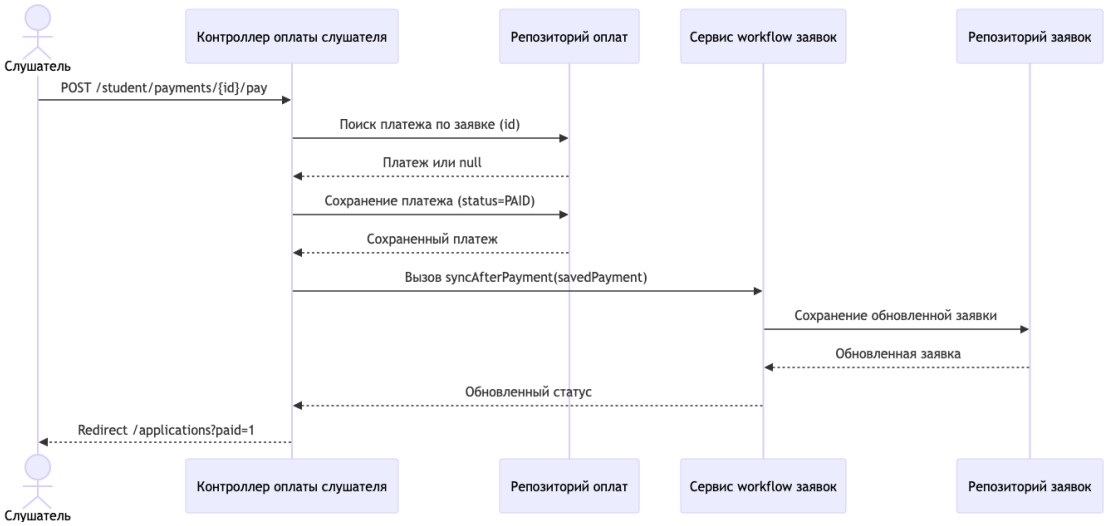

Рисунок 8 — UML Sequence (диаграмма последовательностей): оплата и синхронизация статуса

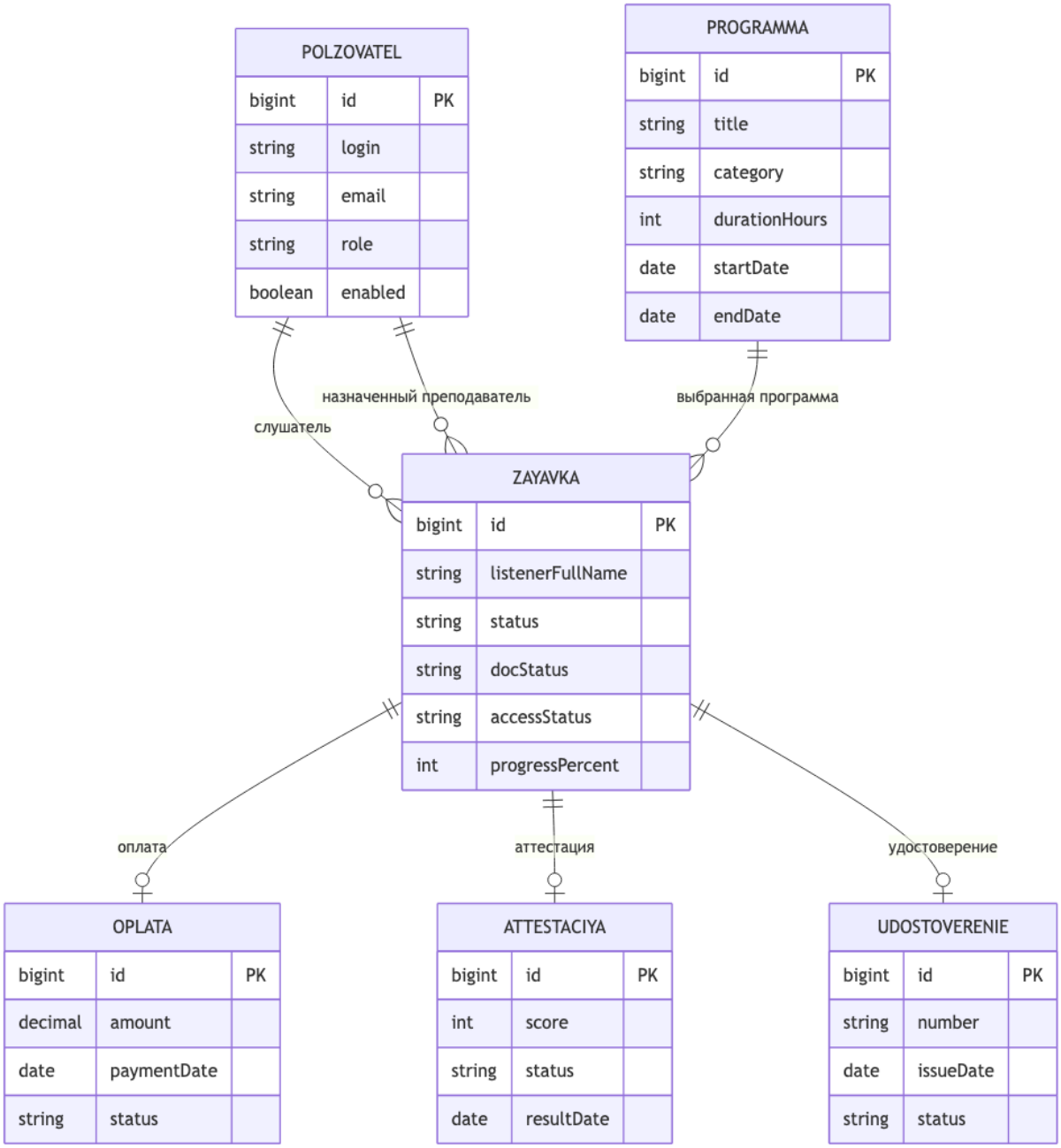

Рисунок 9 — IDEF1X/ER: модель данных системы (сущности и связи)

Приложение Д. Скриншоты системы

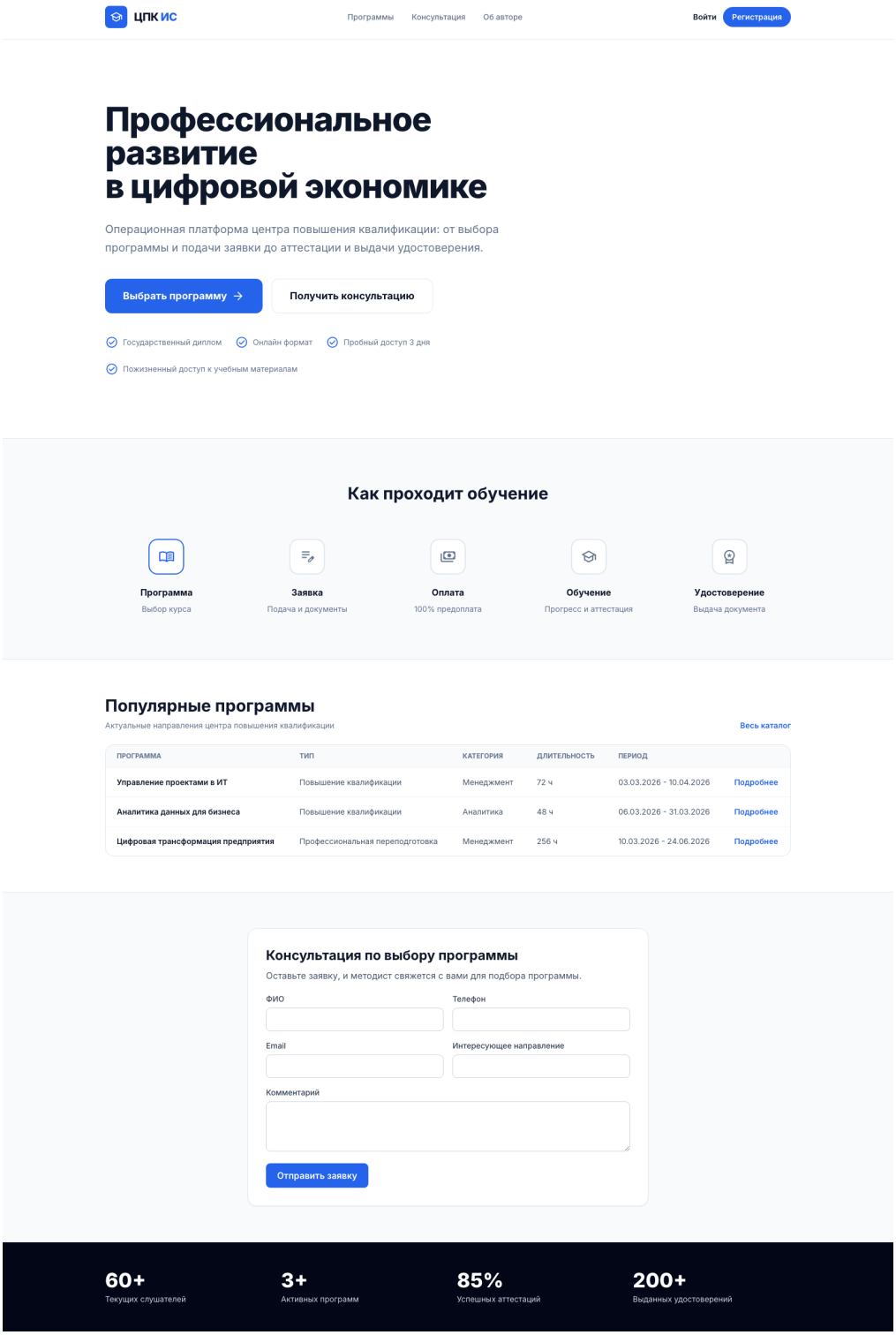

Рисунок 10 — Главная публичная страница

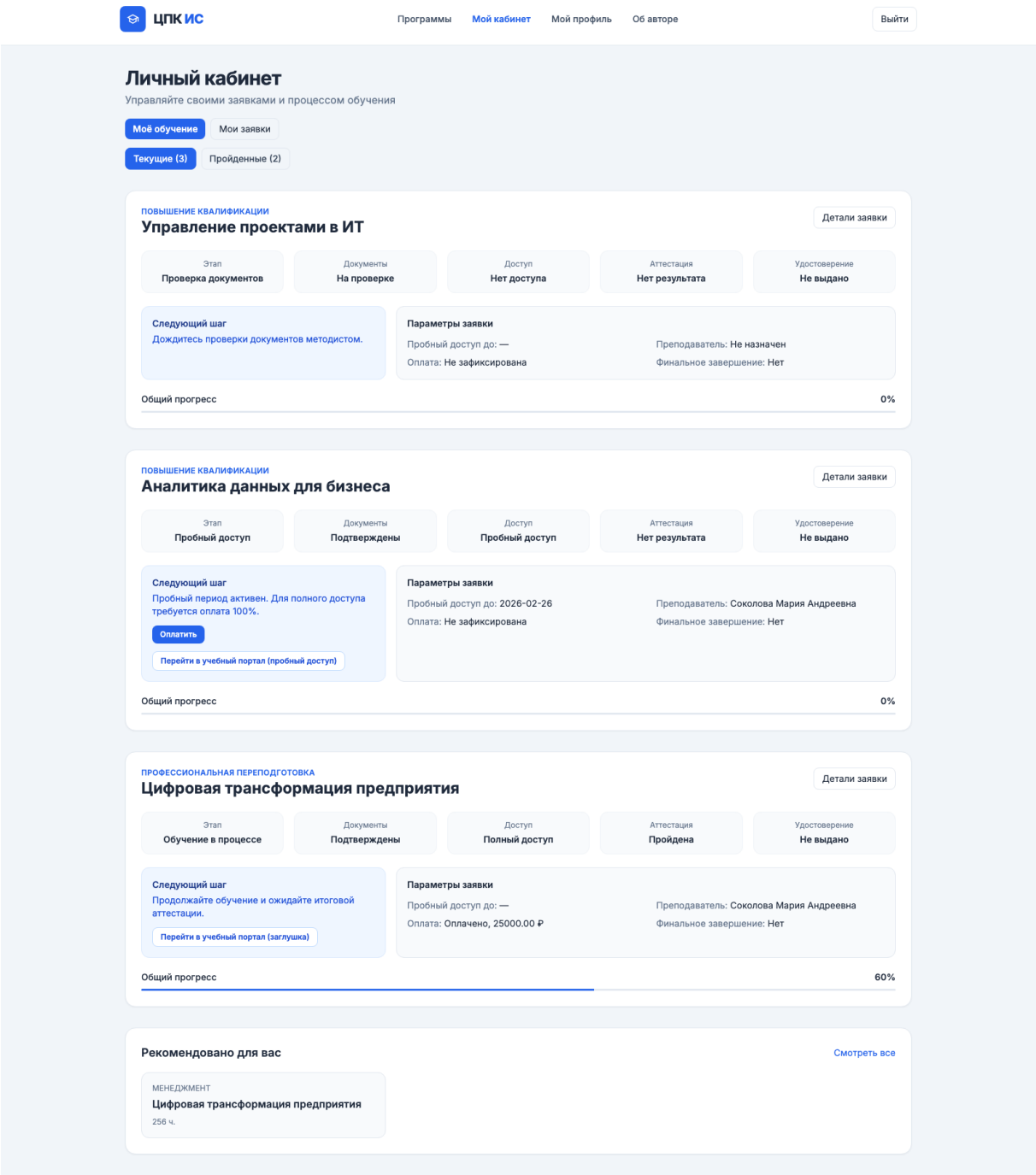

Рисунок 11 — Личный кабинет слушателя

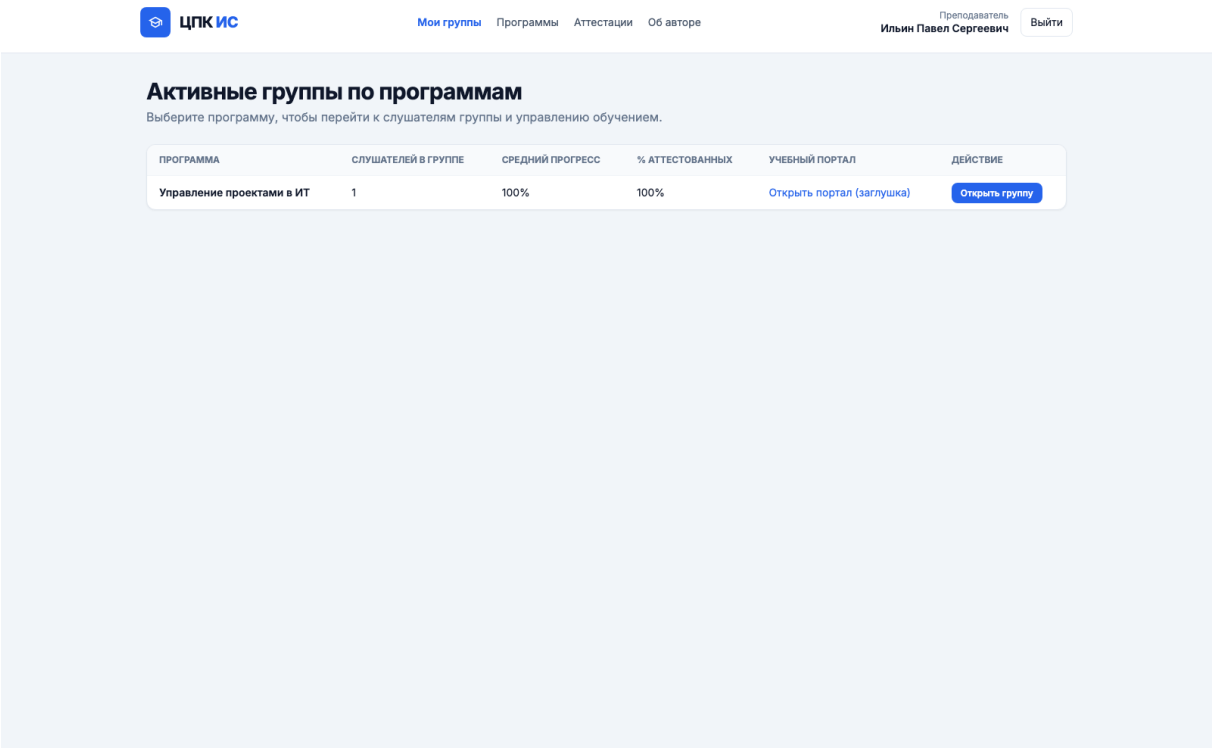

Рисунок 12 — Кабинет преподавателя: активные группы

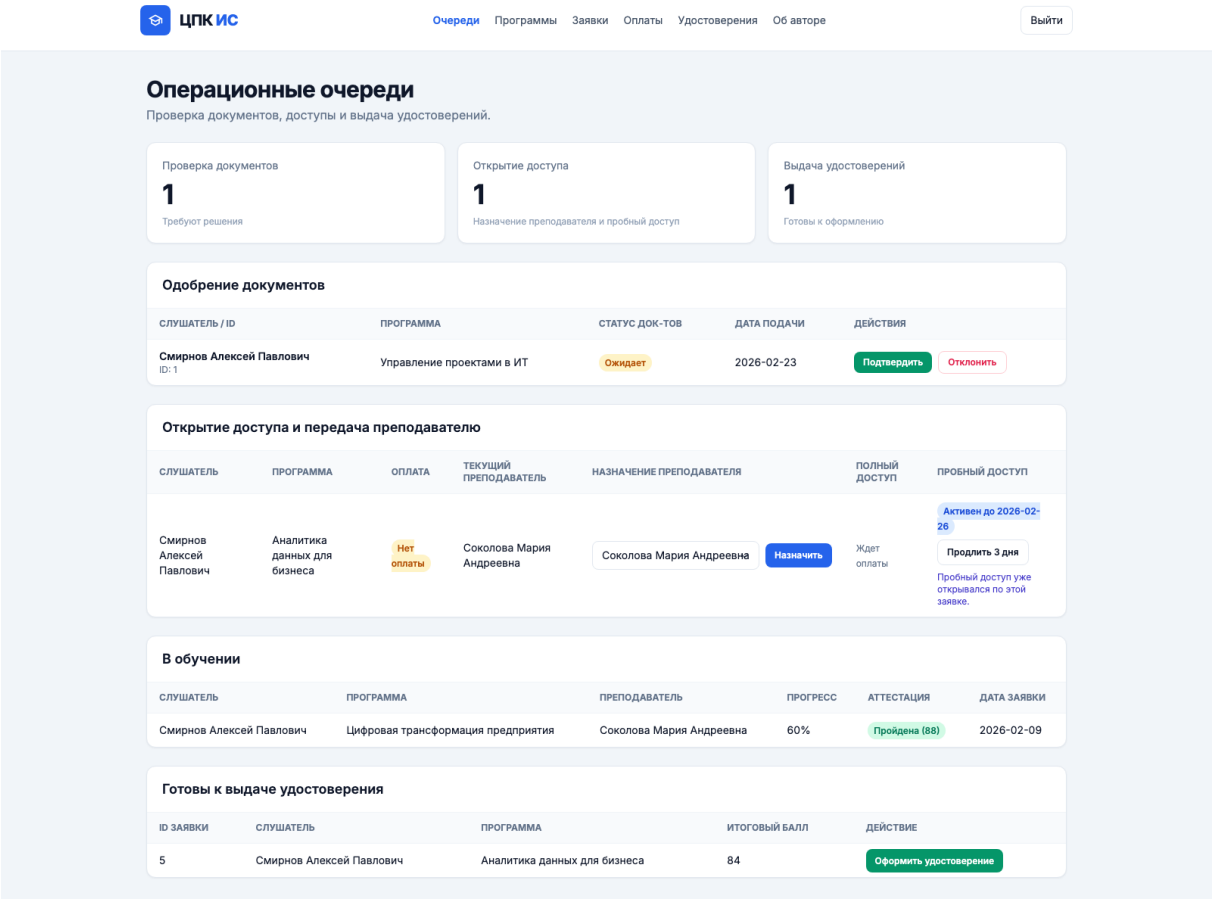

Рисунок 13 — Очередь методиста

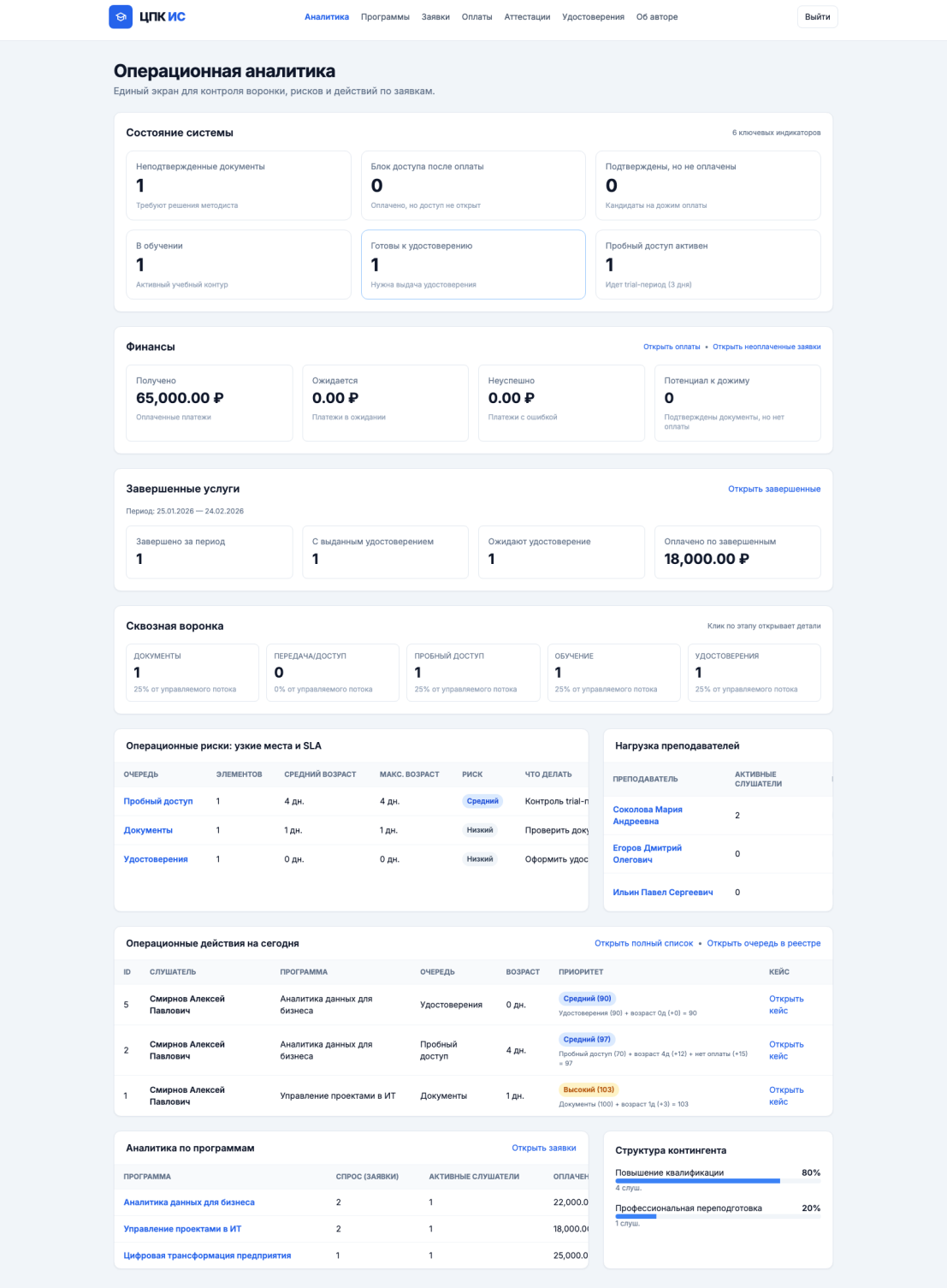

Рисунок 14 — Административная панель dashboard-v2